Paso 1 — Crear las 25 fechas mensuales

```
DBI::dbExecute(sc, "
DROP TABLE IF EXISTS bu_frd.design_a_500k_2023_month_keys_25m
")


DBI::dbExecute(sc, "
CREATE TABLE bu_frd.design_a_500k_2023_month_keys_25m
USING PARQUET
AS

SELECT
   s.party_id,
   s.observation_date,
   s.label_3m,

   x.month_offset,

   LAST_DAY(
      ADD_MONTHS(
         s.observation_date,
         -x.month_offset
      )
   ) AS data_date,

   CASE
      WHEN x.month_offset = 24 THEN 1
      ELSE 0
   END AS seed_month_flag

FROM bu_frd.design_a_500k_2023 s

CROSS JOIN (
   SELECT EXPLODE(SEQUENCE(0,24)) AS month_offset
) x
")
```

Paso 2 — Extraer trades para esos 25 meses

```
DBI::dbExecute(sc, "
DROP TABLE IF EXISTS bu_frd.design_a_500k_2023_trades_25m
")


DBI::dbExecute(sc, "
```

```
CREATE TABLE bu_frd.design_a_500k_2023_trades_25m
USING PARQUET
AS

SELECT

    k.party_id,
    k.observation_date,
    k.label_3m,
    k.month_offset,
    k.data_date,
    k.seed_month_flag,

    t.acct_id,
    t.acct_num,
    t.acct_typ_cde,
    t.porf_typ_cde,

    t.kob_posn_1_cde,
    t.kob_posn_2_cde,

    t.open_dte,
    t.clsd_dte,
    t.paid_out_dte,

    t.eff_dte,
    t.acct_mop_cde,

    t.curr_bal_amt,
    t.cr_lmt_amt,
    t.high_cr_amt,
    t.past_due_amt,

    t.payt_amt,
    t.payt_dte

FROM bu_frd.design_a_500k_2023_month_keys_25m k

INNER JOIN trades t
    ON k.party_id = t.party_id
   AND k.data_date = t.data_date
")
```

Paso 3 — Reconstruir las transiciones MOP

```
DBI::dbExecute(sc, "
DROP TABLE IF EXISTS bu_frd.design_a_500k_2023_mop_transition_25m
```

```
")

DBI::dbExecute(sc, "
CREATE TABLE bu_frd.design_a_500k_2023_mop_transition_25m
USING PARQUET
AS

WITH base AS (

    SELECT

        *,

        LAG(acct_mop_cde) OVER (
            PARTITION BY party_id, acct_id
            ORDER BY data_date
        ) AS previous_mop,

        LAG(eff_dte) OVER (
            PARTITION BY party_id, acct_id
            ORDER BY data_date
        ) AS previous_eff_dte,

        LAG(curr_bal_amt) OVER (
            PARTITION BY party_id, acct_id
            ORDER BY data_date
        ) AS previous_balance,

        LAG(cr_lmt_amt) OVER (
            PARTITION BY party_id, acct_id
            ORDER BY data_date
        ) AS previous_credit_limit,

        LAG(past_due_amt) OVER (
            PARTITION BY party_id, acct_id
            ORDER BY data_date
        ) AS previous_past_due

    FROM bu_frd.design_a_500k_2023_trades_25m
),

mapped AS (

    SELECT
        *,

        CASE
            WHEN previous_mop = '01' THEN 0
```

```
            WHEN previous_mop = '02' THEN 1
            WHEN previous_mop = '03' THEN 2
            WHEN previous_mop = '04' THEN 3
            WHEN previous_mop = '05' THEN 4
            WHEN previous_mop = '06' THEN 5
            ELSE NULL
        END AS previous_delinquency_level,

        CASE
            WHEN acct_mop_cde = '01' THEN 0
            WHEN acct_mop_cde = '02' THEN 1
            WHEN acct_mop_cde = '03' THEN 2
            WHEN acct_mop_cde = '04' THEN 3
            WHEN acct_mop_cde = '05' THEN 4
            WHEN acct_mop_cde = '06' THEN 5
            ELSE NULL
        END AS current_delinquency_level

    FROM base
)

SELECT
    *,

    /* Active account */
    CASE
        WHEN (open_dte IS NULL OR open_dte <= data_date)
         AND (clsd_dte IS NULL OR clsd_dte > data_date)
        THEN 1
        ELSE 0
    END AS active_account_flag,


    /* Effective update */
    CASE
        WHEN previous_eff_dte IS NOT NULL
         AND eff_dte <> previous_eff_dte
        THEN 1
        ELSE 0
    END AS effective_update_flag,


    /* Deterioration */
    CASE
        WHEN previous_delinquency_level IS NOT NULL
         AND current_delinquency_level IS NOT NULL
         AND current_delinquency_level >
             previous_delinquency_level
```

```
    THEN 1
    ELSE 0
END AS dpd_deterioration_flag,


/* Improvement */
CASE
    WHEN previous_delinquency_level IS NOT NULL
     AND current_delinquency_level IS NOT NULL
     AND current_delinquency_level <
         previous_delinquency_level
    THEN 1
    ELSE 0
END AS dpd_improvement_flag,


/* Magnitude of ordinal movement */
CASE
    WHEN previous_delinquency_level IS NOT NULL
     AND current_delinquency_level IS NOT NULL
    THEN current_delinquency_level
         - previous_delinquency_level
    ELSE NULL
END AS dpd_level_change,


/* Entered active adverse */
CASE
    WHEN acct_mop_cde IN ('09','9B','8A','8B')
     AND COALESCE(previous_mop,'')
         NOT IN ('09','9B','8A','8B')
    THEN 1
    ELSE 0
END AS entered_active_adverse_flag,


/* Adverse paid/resolved */
CASE
    WHEN acct_mop_cde IN ('9P','8P')
     AND COALESCE(previous_mop,'') <> acct_mop_cde
    THEN 1
    ELSE 0
END AS adverse_resolved_flag,


/* Became unrated */
CASE
    WHEN acct_mop_cde = 'UR'
```

```
        AND COALESCE(previous_mop,'') <> 'UR'
       THEN 1
       ELSE 0
   END AS became_unrated_flag,


   /* Left unrated */
   CASE
      WHEN previous_mop = 'UR'
       AND acct_mop_cde <> 'UR'
      THEN 1
      ELSE 0
   END AS left_unrated_flag

FROM mapped
")


Paso 4 — Agregación mensual por party

DBI::dbExecute(sc, "
DROP TABLE IF EXISTS bu_frd.design_a_500k_2023_behavior_monthly
")


DBI::dbExecute(sc, "
CREATE TABLE bu_frd.design_a_500k_2023_behavior_monthly
USING PARQUET
AS

SELECT

   party_id,
   observation_date,
   label_3m,
   month_offset,
   data_date,
   seed_month_flag,


   /* ========================================================
      PORTFOLIO
      ======================================================== */

   COUNT(DISTINCT acct_id)
      AS total_accounts,

   SUM(active_account_flag)
```

```
   AS active_accounts,

SUM(
   CASE
      WHEN clsd_dte IS NOT NULL
       AND clsd_dte <= data_date
      THEN 1
      ELSE 0
   END
) AS closed_accounts,


/* ============================================================
  EXPOSURE
  ============================================================ */

SUM(
   CASE
      WHEN active_account_flag = 1
      THEN COALESCE(curr_bal_amt,0)
      ELSE 0
   END
) AS total_active_balance,

SUM(
   CASE
      WHEN active_account_flag = 1
      THEN COALESCE(past_due_amt,0)
      ELSE 0
   END
) AS total_past_due_amount,


/* ============================================================
  DELINQUENCY
  ============================================================ */

SUM(
   CASE
      WHEN active_account_flag = 1
       AND acct_mop_cde IN ('02','03','04','05','06')
      THEN 1
      ELSE 0
   END
) AS delinquent_accounts,

SUM(
   CASE
```

```
        WHEN active_account_flag = 1
         AND acct_mop_cde IN ('03','04','05','06')
        THEN 1
        ELSE 0
    END
) AS dpd_60plus_accounts,

SUM(
    CASE
        WHEN active_account_flag = 1
         AND acct_mop_cde IN ('04','05','06')
        THEN 1
        ELSE 0
    END
) AS dpd_90plus_accounts,


/* ============================================================
   BEHAVIORAL EVENTS
   ============================================================ */

SUM(
    CASE
        WHEN dpd_deterioration_flag = 1
         AND effective_update_flag = 1
        THEN 1
        ELSE 0
    END
) AS deteriorated_accounts,

SUM(
    CASE
        WHEN dpd_improvement_flag = 1
         AND effective_update_flag = 1
        THEN 1
        ELSE 0
    END
) AS improved_accounts,

SUM(
    CASE
        WHEN entered_active_adverse_flag = 1
         AND effective_update_flag = 1
        THEN 1
        ELSE 0
    END
) AS entered_active_adverse_accounts,
```

```
SUM(
   CASE
      WHEN adverse_resolved_flag = 1
       AND effective_update_flag = 1
      THEN 1
      ELSE 0
   END
) AS adverse_resolved_accounts,

SUM(
   CASE
      WHEN became_unrated_flag = 1
       AND effective_update_flag = 1
      THEN 1
      ELSE 0
   END
) AS became_unrated_accounts,

SUM(
   CASE
      WHEN left_unrated_flag = 1
       AND effective_update_flag = 1
      THEN 1
      ELSE 0
   END
) AS left_unrated_accounts,


/* ============================================================
  SEVERITY
  ============================================================ */

MAX(
   CASE
      WHEN dpd_deterioration_flag = 1
       AND effective_update_flag = 1
      THEN dpd_level_change
   END
) AS max_dpd_jump,

SUM(
   CASE
      WHEN dpd_deterioration_flag = 1
       AND effective_update_flag = 1
      THEN dpd_level_change
      ELSE 0
   END
) AS total_dpd_deterioration_points,
```

```
    /* ============================================================
       FRESHNESS
       ============================================================ */

    COUNT(
       DISTINCT CASE
          WHEN effective_update_flag = 1
          THEN acct_id
       END
    ) AS effectively_updated_accounts,


    /* ============================================================
       REVOLVING / LINE OF CREDIT
       ============================================================ */

    SUM(
       CASE
          WHEN active_account_flag = 1
           AND porf_typ_cde IN ('R','C')
          THEN COALESCE(curr_bal_amt,0)
          ELSE 0
       END
    ) AS revolving_balance,

    SUM(
       CASE
          WHEN active_account_flag = 1
           AND porf_typ_cde IN ('R','C')
          THEN COALESCE(cr_lmt_amt,0)
          ELSE 0
       END
    ) AS revolving_credit_limit

FROM bu_frd.design_a_500k_2023_mop_transition_25m

GROUP BY
    party_id,
    observation_date,
    label_3m,
    month_offset,
    data_date,
    seed_month_flag
")
```

Paso 5 — Crear métricas mensuales derivadas

```
DBI::dbExecute(sc, "
DROP TABLE IF EXISTS bu_frd.design_a_500k_2023_behavior_monthly_v2
")


DBI::dbExecute(sc, "
CREATE TABLE bu_frd.design_a_500k_2023_behavior_monthly_v2
USING PARQUET
AS

SELECT
  *,

  /* Average exposure per active account */

  CASE
    WHEN active_accounts > 0
    THEN 1.0 * total_active_balance / active_accounts
  END AS avg_active_balance,

  CASE
    WHEN active_accounts > 0
    THEN 1.0 * total_past_due_amount / active_accounts
  END AS avg_past_due_active,


  /* Delinquency shares */

  CASE
    WHEN active_accounts > 0
    THEN 1.0 * delinquent_accounts / active_accounts
  END AS pct_accounts_delinquent,

  CASE
    WHEN active_accounts > 0
    THEN 1.0 * dpd_60plus_accounts / active_accounts
  END AS pct_accounts_60plus,

  CASE
    WHEN active_accounts > 0
    THEN 1.0 * dpd_90plus_accounts / active_accounts
  END AS pct_accounts_90plus,


  /* Behavioral shares */
```

```
    CASE
      WHEN active_accounts > 0
      THEN 1.0 * deteriorated_accounts / active_accounts
    END AS pct_accounts_deteriorated,

    CASE
      WHEN active_accounts > 0
      THEN 1.0 * entered_active_adverse_accounts
          / active_accounts
    END AS pct_accounts_entered_adverse,

    CASE
      WHEN active_accounts > 0
      THEN 1.0 * effectively_updated_accounts
          / active_accounts
    END AS pct_accounts_effectively_updated,


    /* Revolving utilization */

    CASE
      WHEN revolving_credit_limit > 0
      THEN 1.0 * revolving_balance
          / revolving_credit_limit
    END AS revolving_utilization

FROM bu_frd.design_a_500k_2023_behavior_monthly
")


Paso 6 — Crear directamente las ventanas 24M / 12M / 6M / 3M

DBI::dbExecute(sc, "
DROP TABLE IF EXISTS bu_frd.design_a_500k_2023_features_24_12_6_3m
")


DBI::dbExecute(sc, "
CREATE TABLE bu_frd.design_a_500k_2023_features_24_12_6_3m
USING PARQUET
AS

SELECT

    party_id,

    MAX(observation_date)
      AS observation_date,
```

```
MAX(label_3m)
   AS label_3m,


/* ========================================================
  ACTIVE ACCOUNTS
  ======================================================== */

AVG(CASE WHEN month_offset BETWEEN 0 AND 23
   THEN active_accounts END)
   AS active_accounts_mean_24m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 11
   THEN active_accounts END)
   AS active_accounts_mean_12m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 5
   THEN active_accounts END)
   AS active_accounts_mean_6m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 2
   THEN active_accounts END)
   AS active_accounts_mean_3m,


/* ========================================================
  ACTIVE BALANCE
  ======================================================== */

AVG(CASE WHEN month_offset BETWEEN 0 AND 23
   THEN avg_active_balance END)
   AS avg_active_balance_24m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 11
   THEN avg_active_balance END)
   AS avg_active_balance_12m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 5
   THEN avg_active_balance END)
   AS avg_active_balance_6m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 2
   THEN avg_active_balance END)
   AS avg_active_balance_3m,


/* ========================================================
```

```
  PAST DUE
  ========================================================== */

AVG(CASE WHEN month_offset BETWEEN 0 AND 23
   THEN avg_past_due_active END)
   AS avg_past_due_active_24m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 11
   THEN avg_past_due_active END)
   AS avg_past_due_active_12m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 5
   THEN avg_past_due_active END)
   AS avg_past_due_active_6m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 2
   THEN avg_past_due_active END)
   AS avg_past_due_active_3m,


/* ==========================================================
  DELINQUENCY SHARE
  ========================================================== */

AVG(CASE WHEN month_offset BETWEEN 0 AND 23
   THEN pct_accounts_delinquent END)
   AS delinquency_share_24m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 11
   THEN pct_accounts_delinquent END)
   AS delinquency_share_12m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 5
   THEN pct_accounts_delinquent END)
   AS delinquency_share_6m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 2
   THEN pct_accounts_delinquent END)
   AS delinquency_share_3m,


/* ==========================================================
  REVOLVING UTILIZATION
  ========================================================== */

AVG(CASE WHEN month_offset BETWEEN 0 AND 23
   THEN revolving_utilization END)
   AS revolving_utilization_24m,
```

```
AVG(CASE WHEN month_offset BETWEEN 0 AND 11
    THEN revolving_utilization END)
    AS revolving_utilization_12m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 5
    THEN revolving_utilization END)
    AS revolving_utilization_6m,

AVG(CASE WHEN month_offset BETWEEN 0 AND 2
    THEN revolving_utilization END)
    AS revolving_utilization_3m,


/* ============================================================
   DETERIORATION EVENTS
   ============================================================ */

SUM(CASE WHEN month_offset BETWEEN 0 AND 23
    THEN deteriorated_accounts ELSE 0 END)
    AS deterioration_count_24m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 11
    THEN deteriorated_accounts ELSE 0 END)
    AS deterioration_count_12m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 5
    THEN deteriorated_accounts ELSE 0 END)
    AS deterioration_count_6m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 2
    THEN deteriorated_accounts ELSE 0 END)
    AS deterioration_count_3m,


/* ============================================================
   IMPROVEMENT EVENTS
   ============================================================ */

SUM(CASE WHEN month_offset BETWEEN 0 AND 23
    THEN improved_accounts ELSE 0 END)
    AS improvement_count_24m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 11
    THEN improved_accounts ELSE 0 END)
    AS improvement_count_12m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 5
```

```
    THEN improved_accounts ELSE 0 END)
    AS improvement_count_6m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 2
    THEN improved_accounts ELSE 0 END)
    AS improvement_count_3m,


/* ========================================================
   ADVERSE ENTRY
   ======================================================== */

SUM(CASE WHEN month_offset BETWEEN 0 AND 23
    THEN entered_active_adverse_accounts ELSE 0 END)
    AS adverse_entry_count_24m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 11
    THEN entered_active_adverse_accounts ELSE 0 END)
    AS adverse_entry_count_12m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 5
    THEN entered_active_adverse_accounts ELSE 0 END)
    AS adverse_entry_count_6m,

SUM(CASE WHEN month_offset BETWEEN 0 AND 2
    THEN entered_active_adverse_accounts ELSE 0 END)
    AS adverse_entry_count_3m,


/* ========================================================
   MAXIMUM DPD JUMP
   ======================================================== */

MAX(CASE WHEN month_offset BETWEEN 0 AND 23
    THEN max_dpd_jump END)
    AS max_dpd_jump_24m,

MAX(CASE WHEN month_offset BETWEEN 0 AND 11
    THEN max_dpd_jump END)
    AS max_dpd_jump_12m,

MAX(CASE WHEN month_offset BETWEEN 0 AND 5
    THEN max_dpd_jump END)
    AS max_dpd_jump_6m,

MAX(CASE WHEN month_offset BETWEEN 0 AND 2
    THEN max_dpd_jump END)
    AS max_dpd_jump_3m,
```

```
    /* ============================================================
       MONTHS AVAILABLE
       ============================================================ */

    COUNT(CASE WHEN month_offset BETWEEN 0 AND 23
               AND active_accounts > 0
          THEN 1 END)
        AS active_months_available_24m,

    COUNT(CASE WHEN month_offset BETWEEN 0 AND 11
               AND active_accounts > 0
          THEN 1 END)
        AS active_months_available_12m,

    COUNT(CASE WHEN month_offset BETWEEN 0 AND 5
               AND active_accounts > 0
          THEN 1 END)
        AS active_months_available_6m,

    COUNT(CASE WHEN month_offset BETWEEN 0 AND 2
               AND active_accounts > 0
          THEN 1 END)
        AS active_months_available_3m

FROM bu_frd.design_a_500k_2023_behavior_monthly_v2

WHERE month_offset BETWEEN 0 AND 23

GROUP BY party_id
")
```